OneLake User Guide

Browse, read, write, download, and upload Lakehouse files from Zone notebooks

What this guide helps users do

- Connect a notebook to any Fabric workspace and Lakehouse
- Understand Files, Tables, and OneLake paths
- Use every onelake command with practical examples
- Use the same access from terminal, Python, R, and JupyterLab

Token model: access tokens stay in memory only; saved config is workspace/lakehouse only.

Branch for testing: feat/onelake-personal-drive | Image tag: feat-onelake-personal-drive

Start Here

> **Plain-English Summary**
> onelake is the notebook command for working with Fabric Lakehouse files. Users connect once to a workspace and Lakehouse, then list, read, write, download, and upload files using their own delegated permissions.

Use case	What to run
Connect to a Lakehouse	onelake connect <workspace> <lakehouse>
Check setup	onelake status
Test live access	onelake status --live
Browse files	onelake ls Files/
Read small text	onelake cat Files/example.txt
Write text	echo hello | onelake write Files/hello.txt
Download	onelake get Files/input.csv /tmp/input.csv
Upload	onelake put /tmp/output.csv Files/output.csv

How Authentication Works

You do not paste or save a OneLake access token. The notebook asks AuthService for a delegated token when it needs to talk to OneLake.

- The token follows your signed-in user permissions.
- The token is kept in memory only.
- When the notebook stops, the token is gone.
- Your workspace and Lakehouse selection remain saved, so the next command asks AuthService for a fresh token.

> **Security Note**
> onelake saves workspace/lakehouse configuration. It does not save OneLake access tokens, refresh tokens, client secrets, or passwords.

Find Your Workspace And Lakehouse

If you have a Fabric Lakehouse URL, the IDs are in the URL.

```
https://app.fabric.microsoft.com/groups/<workspace-id>/lakehouses/<lakehouse-id>
```

Example from the test Lakehouse:

Field	Value
Workspace	29579e7a-73a2-4ebb-bb49-5725fb50a153
Lakehouse	7b1c997b-5a37-4f12-9174-ccb242c0cc61

First-Time Setup

Run this once in a JupyterLab terminal. Replace the values with your workspace and Lakehouse.

```
onelake connect <workspace-id-or-name> <lakehouse-id-or-name>
```

Example:

```
onelake connect 29579e7a-73a2-4ebb-bb49-5725fb50a153 7b1c997b-5a37-4f12-9174-ccb242c0cc61
```

Expected Output
OneLake configuration saved.

Check That It Works

onelake status

Show saved workspace, Lakehouse, endpoint, broker URL, and token storage model.

```
onelake status
```

onelake status --live

Test the real path: AuthService token request plus OneLake list access.

```
onelake status --live
```

Success looks like: Live: OK - <number> entries under Files

Understand OneLake Paths

onelake uses a simple root with two top-level locations.

```
/
  Files/
  Tables/
```

Important
onelake ls / only shows Files and Tables. To see Lakehouse files, run onelake ls Files/.

Path	Meaning
/	Virtual root created by onelake
Files/	Fabric Lakehouse Files area
Tables/	Fabric Lakehouse Tables area
raw/input.csv	Shortcut for Files/raw/input.csv
Files/raw/input.csv	Explicit file path under Files

Command Reference

onelake connect

Save the workspace and Lakehouse you want to use.

```
onelake connect <workspace> <lakehouse>
```

Use this when setting up for the first time or switching Lakehouses.

onelake status

Show current configuration.

```
onelake status
onelake status --live
```

Use --live to call AuthService and OneLake.

onelake ls

List files and folders.

```
onelake ls /
onelake ls Files/
onelake ls Files/my-folder/
onelake ls Tables/
```

onelake cat

Print a small text file to the terminal.

```
onelake cat Files/readme.txt
```

Use get for large or binary files.

onelake write

Write standard input to a OneLake file. Existing files are overwritten.

```
echo "hello" | onelake write Files/hello.txt
```

onelake get

Download a OneLake file into the notebook container.

```
onelake get Files/data/input.csv /tmp/input.csv
```

Use this when a tool needs a local file path.

onelake put

Upload a local file to OneLake.

```
onelake put /tmp/result.csv Files/results/result.csv
```

Common Workflows

Browse A Lakehouse

```
onelake ls /
onelake ls Files/
onelake ls Tables/
```

Create And Read A Text File

```
echo "created from my notebook" | onelake write Files/my-note.txt
onelake cat Files/my-note.txt
```

Download, Process Locally, Upload Result

```
onelake get Files/raw/input.csv /tmp/input.csv
python process.py /tmp/input.csv /tmp/output.csv
onelake put /tmp/output.csv Files/processed/output.csv
```

Work With Binary Files

```
onelake get Files/report.xlsx /tmp/report.xlsx
onelake put /tmp/report.xlsx Files/report-copy.xlsx
```

> **Tip**
> Do not use cat for binary files. Use get and put.

Python Usage

```
import onelake

onelake.status()
onelake.ls("Files/")

text = onelake.read("Files/hello.txt", text=True)
print(text)

onelake.write("Files/python-output.txt", "hello from Python\n")
onelake.download("Files/data/input.csv", "/tmp/input.csv")
onelake.upload("/tmp/result.csv", "Files/results/result.csv")
```

Python uses the same saved workspace and Lakehouse as the CLI.

R Usage

```
library(onelake)

ol_status()
ol_ls("Files/")

text <- ol_read_text("Files/hello.txt")
cat(text)

ol_write_text("Files/r-output.txt", "hello from R\n")
ol_download("Files/data/input.csv", "/tmp/input.csv")
ol_upload("/tmp/result.csv", "Files/results/result.csv")
```

R shells out to the onelake CLI, so it uses the same saved workspace and Lakehouse.

JupyterLab Drive

Open the OneLake drive in the JupyterLab file browser. You should see Files and Tables. Open Files to browse Lakehouse files.

> **Not A Linux Mount**
> The JupyterLab drive is backed by OneLake APIs. For tools that need real local paths, use onelake get and onelake put.

Test In Any Lakehouse

Use your own workspace and Lakehouse IDs.

```
onelake connect <workspace-id-or-name> <lakehouse-id-or-name>
onelake status --live
onelake ls Files/
```

Create a safe test file:

```
TEST_FILE="Files/onelake-test-$(date +%Y%m%d-%H%M%S).txt"
echo "hello from onelake" | onelake write "$TEST_FILE"
onelake cat "$TEST_FILE"
```

Download and upload:

```
onelake get "$TEST_FILE" /tmp/onelake-test.txt
cat /tmp/onelake-test.txt

echo "uploaded from notebook" > /tmp/onelake-upload.txt
onelake put /tmp/onelake-upload.txt Files/onelake-upload.txt
onelake cat Files/onelake-upload.txt
```

Troubleshooting

Problem	What to check
status says not ready	Run onelake connect again with the correct workspace and Lakehouse.
status --live fails	Check AuthService, permissions, and endpoint/region configuration.
ls / only shows Files and Tables	That is correct. Run onelake ls Files/.
Files show in Fabric but not CLI	Confirm the workspace and Lakehouse IDs match the Fabric URL.
Read works but write fails	You likely need write permission for that Lakehouse path.
A tool needs a local path	Use onelake get, run the tool locally, then onelake put.

Quick Cheat Sheet

```
onelake connect <workspace> <lakehouse>
onelake status
onelake status --live
onelake ls /
onelake ls Files/
onelake ls Tables/
onelake cat Files/example.txt
echo "hello" | onelake write Files/hello.txt
onelake get Files/hello.txt /tmp/hello.txt
onelake put /tmp/hello.txt Files/hello-copy.txt
```